Nectar Network — STRIDE Threat Model & Dataflow Diagrams

Frozen scope: tag `audit-freeze-v1` (commit `dbf0e5cd0a0c…`) — NectarVault + KeeperRegistry, 1,071 functional LOC. **Prepared:** 2026-08-16 for the SCF Soroban Security Audit Bank. **Contents:** Part 1 — dataflow diagrams and trust boundaries (the visual DFDs the threat model is assessed against). Part 2 — the STRIDE threat model. Canonical sources: `docs/security/DATAFLOW.md` and `docs/security/THREAT-MODEL.md` at github.com/Nectar-Network/nectar.

Part 1 — Dataflow & Trust Boundaries

Nectar Network — Data Flow & Trust Boundaries (audit-freeze-v1)

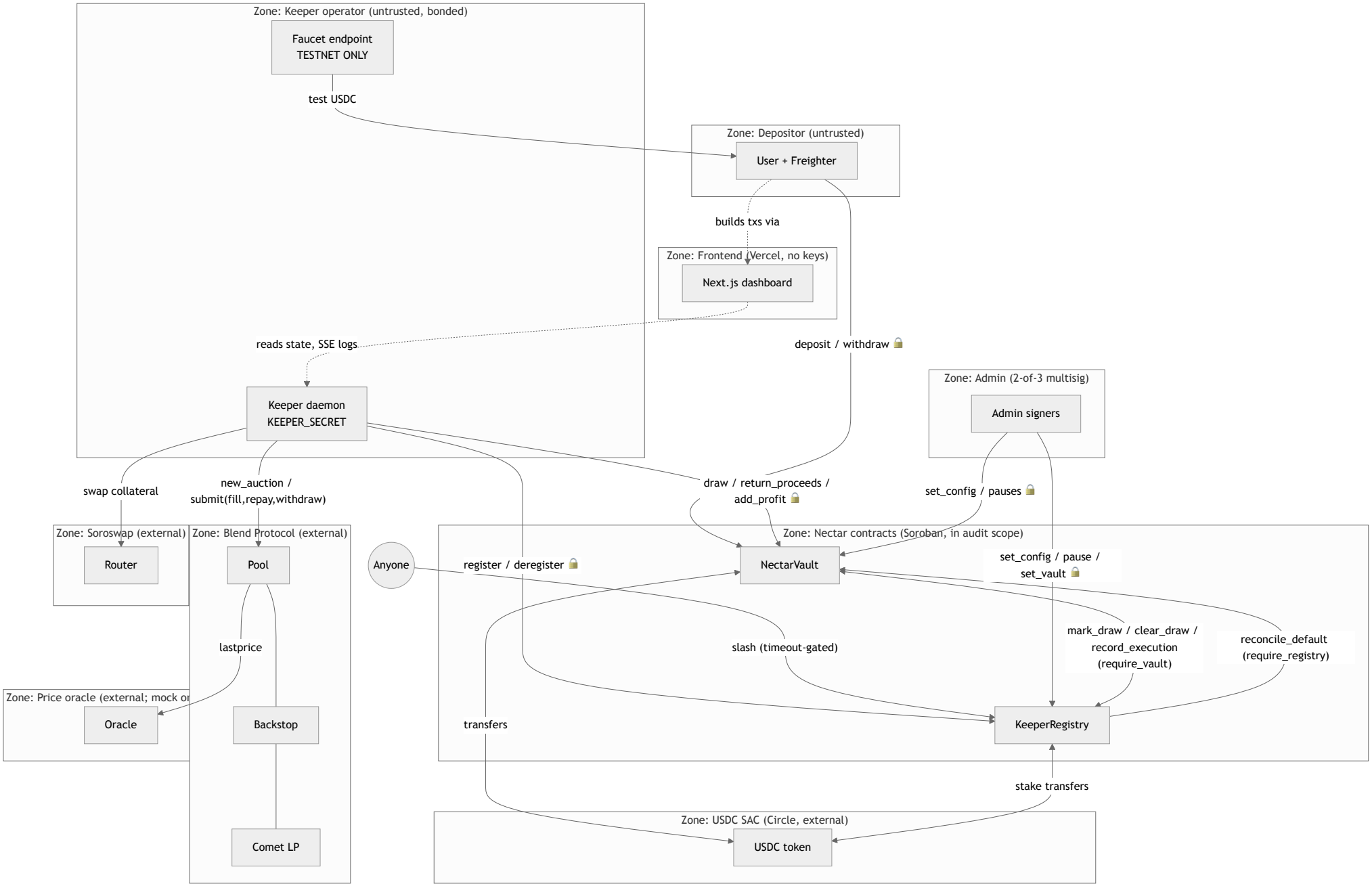
Companion to THREAT-MODEL.md. Every diagram was spot-checked edge-by-edge against the source at tag `audit-freeze-v1` (commit `dbf0e5c`); the liquidation sequence is the one proven live in `docs/evidence/b-full-cycle.md` (tx-hashed), updated for the T3 draw signature — not the pre-verification spec wording.

Line references: `VLT:` = `contracts/nectar-vault/src/lib.rs`, `REG:` = `contracts/keeper-registry/src/lib.rs`, `ADP:` = `keeper/adapters/blend/adapter.go` — all at the frozen tag.

Legend: 🔒 = `require_auth` (signature) · ✅ = on-chain validation · each `subgraph` is a trust zone; every external system is its own zone.

---

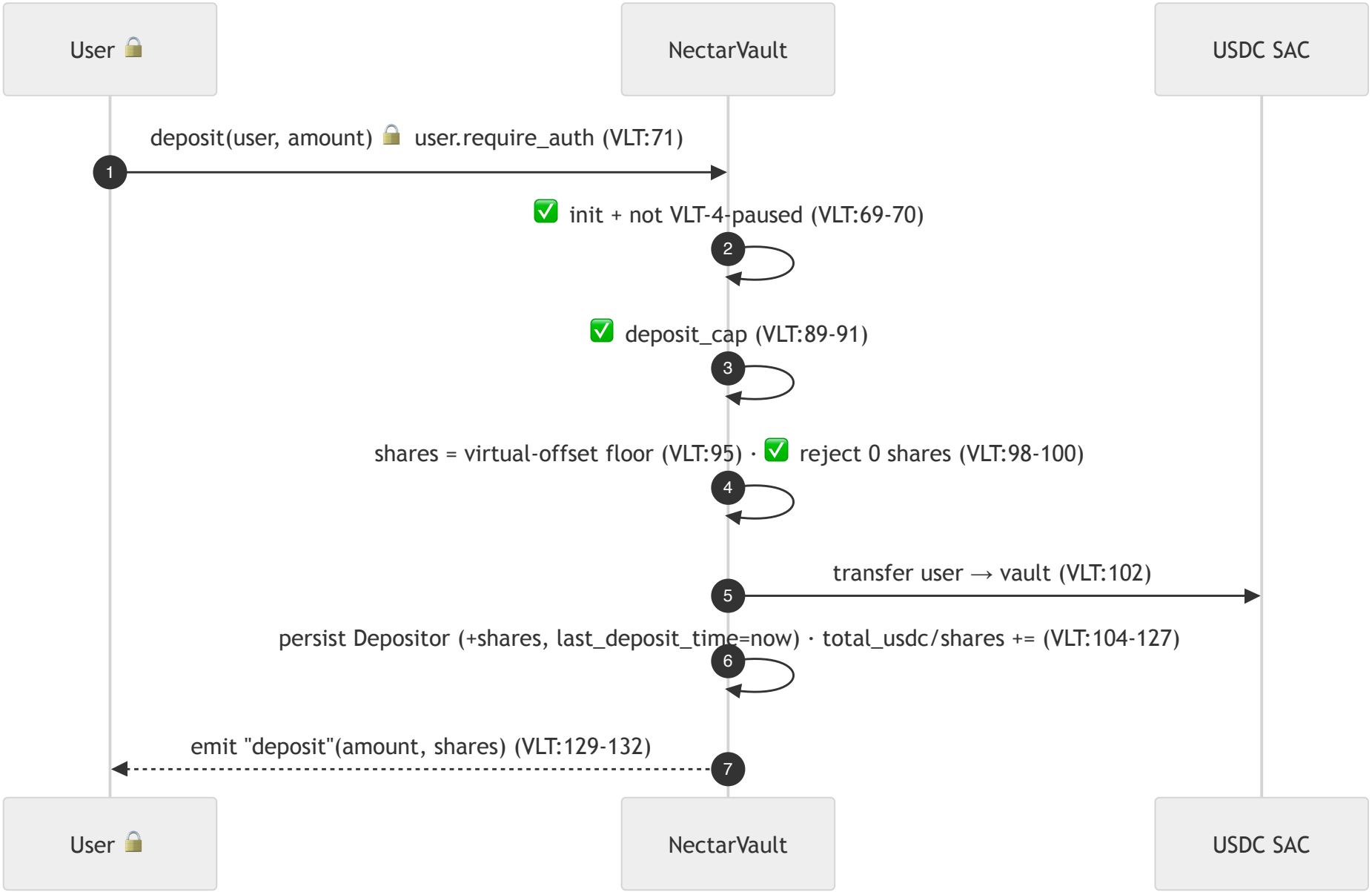
0. Zone map (all trust boundaries at a glance)



The vault and registry pin each other: the registry accepts vault-only calls solely from the address set once via `set_vault` (REG:51-60, 440-451); the vault accepts `reconcile_default` solely from its constructor-stored registry (VLT:758-769). Neither contract ever invokes a caller-supplied address.

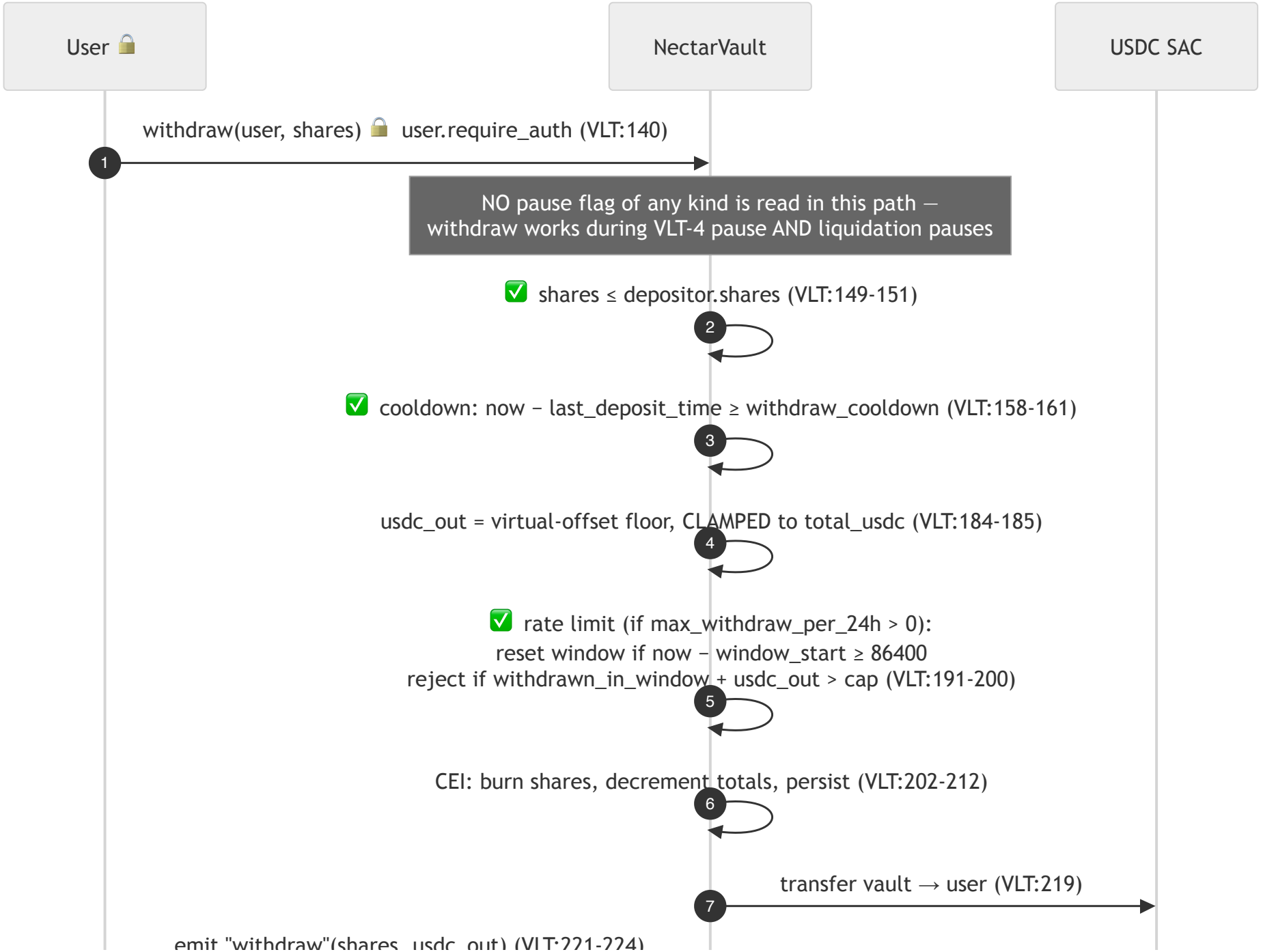
---

1. Deposit



Boundary: B1 depositor→vault. Liquidation pause flags are NOT in this path — deposit is governed by the VLT-4 pause only.

2. Withdraw (cooldown + rate limit in the path)

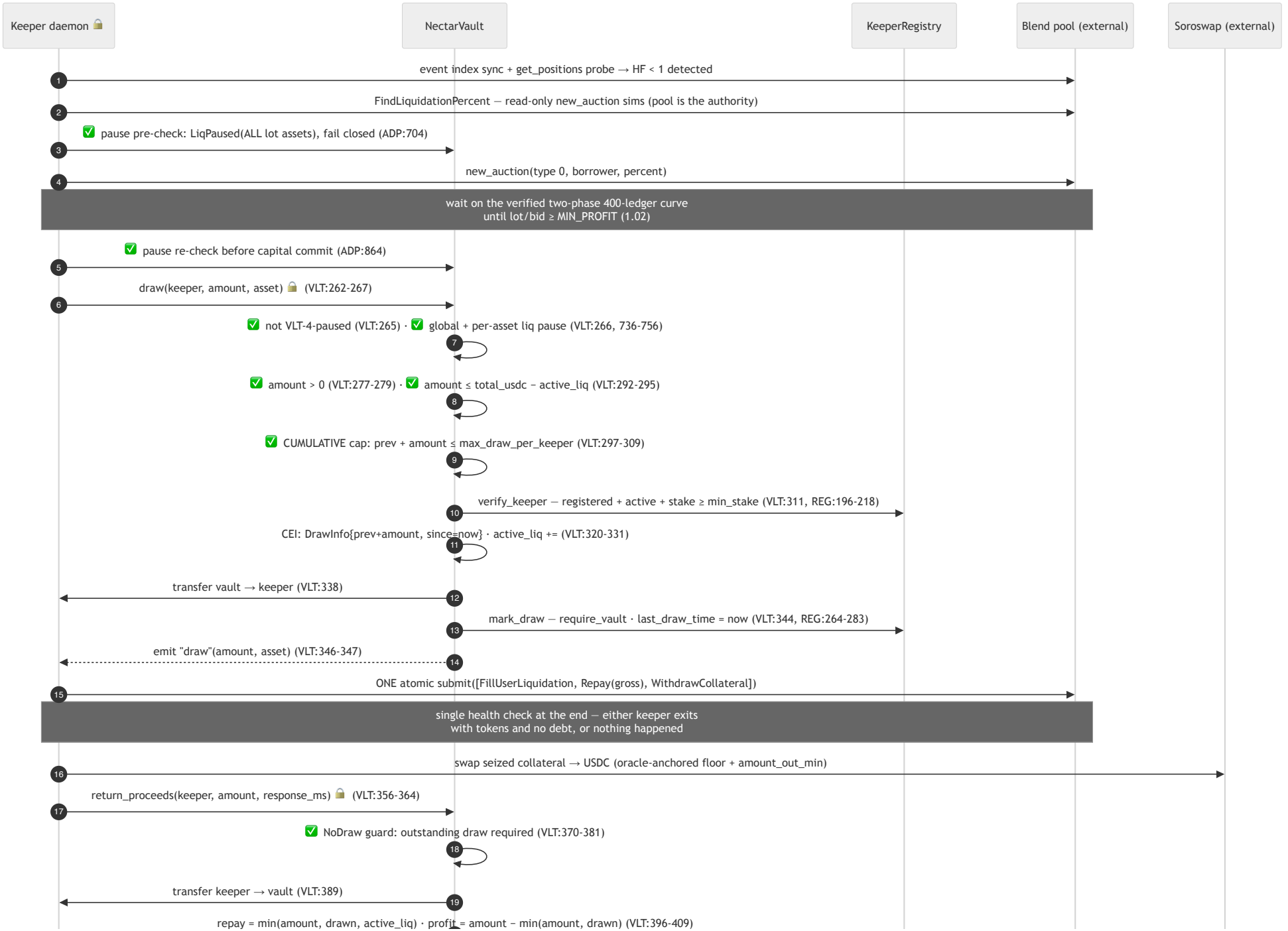

User
NectarVault
USDC SAC
withdraw(user, shares) user.require_auth (VLT:140)
1
NO pause flag of any kind is read in this path —
withdraw works during VLT-4 pause AND liquidation pauses
shares ≤ depositor.shares (VLT:149-151)
2
cooldown: now − last_deposit_time ≥ withdraw_cooldown (VLT:158-161)
3
usdc_out = virtual-offset floor, CLAMPED to total_usdc (VLT:184-185)
4
rate limit (if max_withdraw_per_24h > 0):
reset window if now − window_start ≥ 86400
reject if withdrawn_in_window + usdc_out > cap (VLT:191-200)
5
CEI: burn shares, decrement totals, persist (VLT:202-212)
6
transfer vault → user (VLT:219)
7
emit "withdraw"(shares, usdc_out) (VLT:221-224)

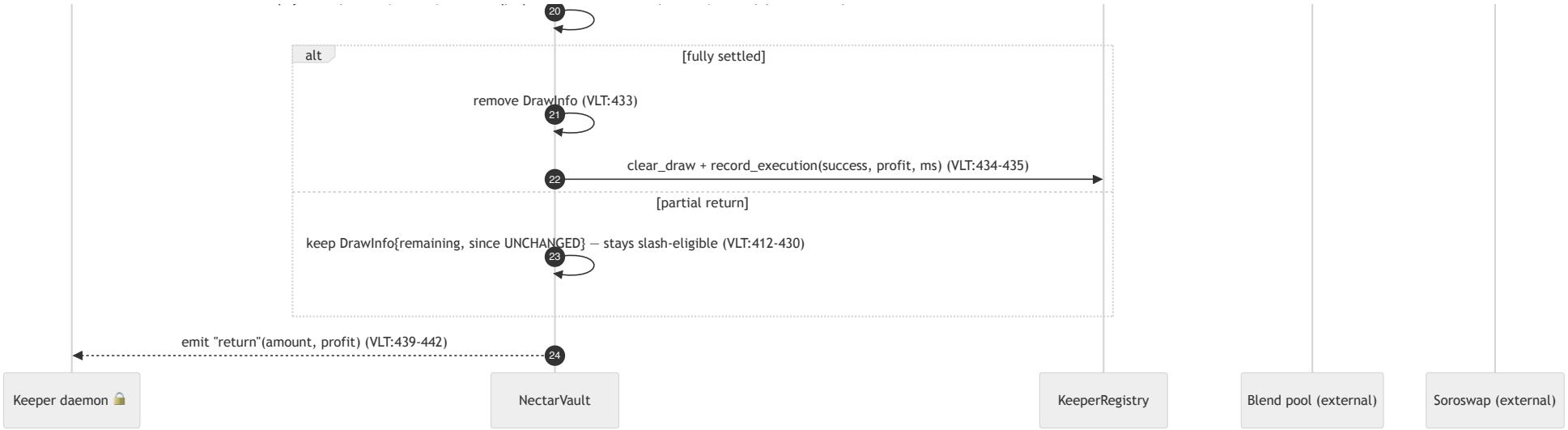
Order matters (all pre-commit): auth → ownership → cooldown → payout computation (with the anti-underflow clamp) → fixed-window rate limit → CEI state commit → token transfer. A rejected withdrawal mutates nothing. The rate limit meters the USDC actually leaving (`usdc_out`), not the share count.

---

3. Full liquidation cycle (as shipped and proven live)

This is the verified sequence from `b-full-cycle.md` (share price 1.0000000 → 1.0432672, every step tx-hashed), with the T3 draw signature at the frozen tag.

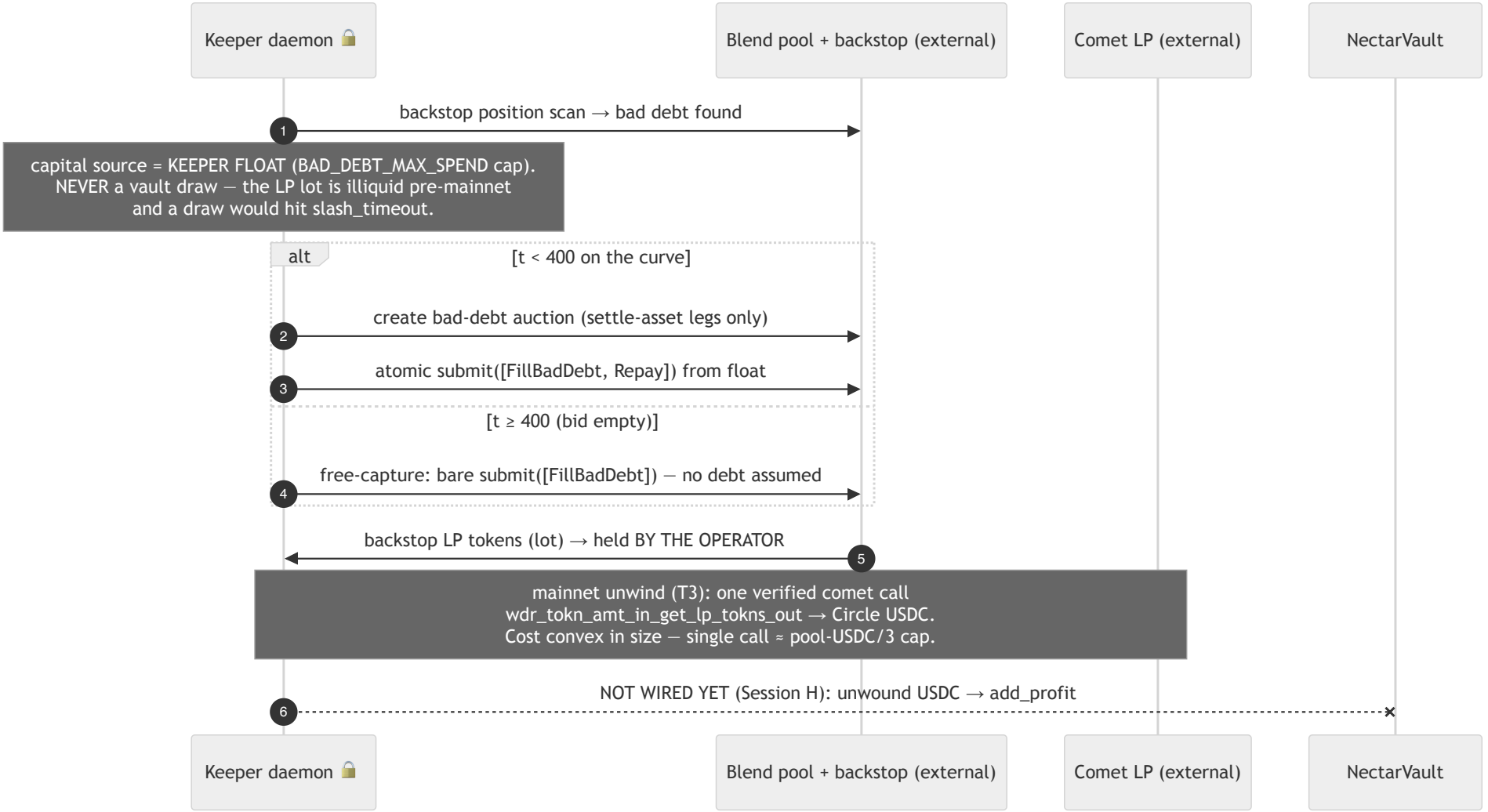

Keeper daemon
NectarVault
KeeperRegistry
Blend pool (external)
Soroswap (external)
1
event index sync + get_positions probe → HF < 1 detected
2
FindLiquidationPercent — read-only new_auction sims (pool is the authority)
3
✅ pause pre-check: LiqPaused(ALL lot assets), fail closed (ADP:704)
4
new_auction(type 0, borrower, percent)
wait on the verified two-phase 400-ledger curve
until lot/bid ≥ MIN_PROFIT (1.02)
5
✅ pause re-check before capital commit (ADP:864)
6
draw(keeper, amount, asset) (VLT:262-267)
✅ not VLT-4-paused (VLT:265) · ✅ global + per-asset liq pause (VLT:266, 736-756)
7
✅ amount > 0 (VLT:277-279) · ✅ amount ≤ total_usdc − active_liq (VLT:292-295)
8
✅ CUMULATIVE cap: prev + amount ≤ max_draw_per_keeper (VLT:297-309)
9
verify_keeper — registered + active + stake ≥ min_stake (VLT:311, REG:196-218)
10
CEI: DrawInfo{prev+amount, since=now} · active_liq += (VLT:320-331)
11
transfer vault → keeper (VLT:338)
12
mark_draw — require_vault · last_draw_time = now (VLT:344, REG:264-283)
13
emit "draw"(amount, asset) (VLT:346-347)
14
15
ONE atomic submit([FillUserLiquidation, Repay(gross), WithdrawCollateral])
single health check at the end — either keeper exits
with tokens and no debt, or nothing happened
16
swap seized collateral → USDC (oracle-anchored floor + amount_out_min)
17
return_proceeds(keeper, amount, response_ms) (VLT:356-364)
✅ NoDraw guard: outstanding draw required (VLT:370-381)
18
transfer keeper → vault (VLT:389)
19
repay = min(amount, drawn, active_liq) · profit = amount − min(amount, drawn) (VLT:396-409)



Boundaries crossed: B5 (keeper↔Blend/Soroswap — nothing those zones return can move vault funds), B2 (keeper↔vault, twice), B3 (vault↔registry, three calls). The vault never talks to Blend, the DEX, or any oracle.

---

4. Bad-debt float path (shipped; operator capital only)



Honest status (CORRECTION-REPORT limitation 2 + Gap 4): at this tag, bad-debt profit accrues to the **operator**, not depositors. The contract-side credit path (`add_profit`, flow 5) exists and is frozen for audit; the keeper-side wiring — unwind → `add_profit` — is Session H work. No document should claim the depositor-credit loop is closed until that wiring ships.

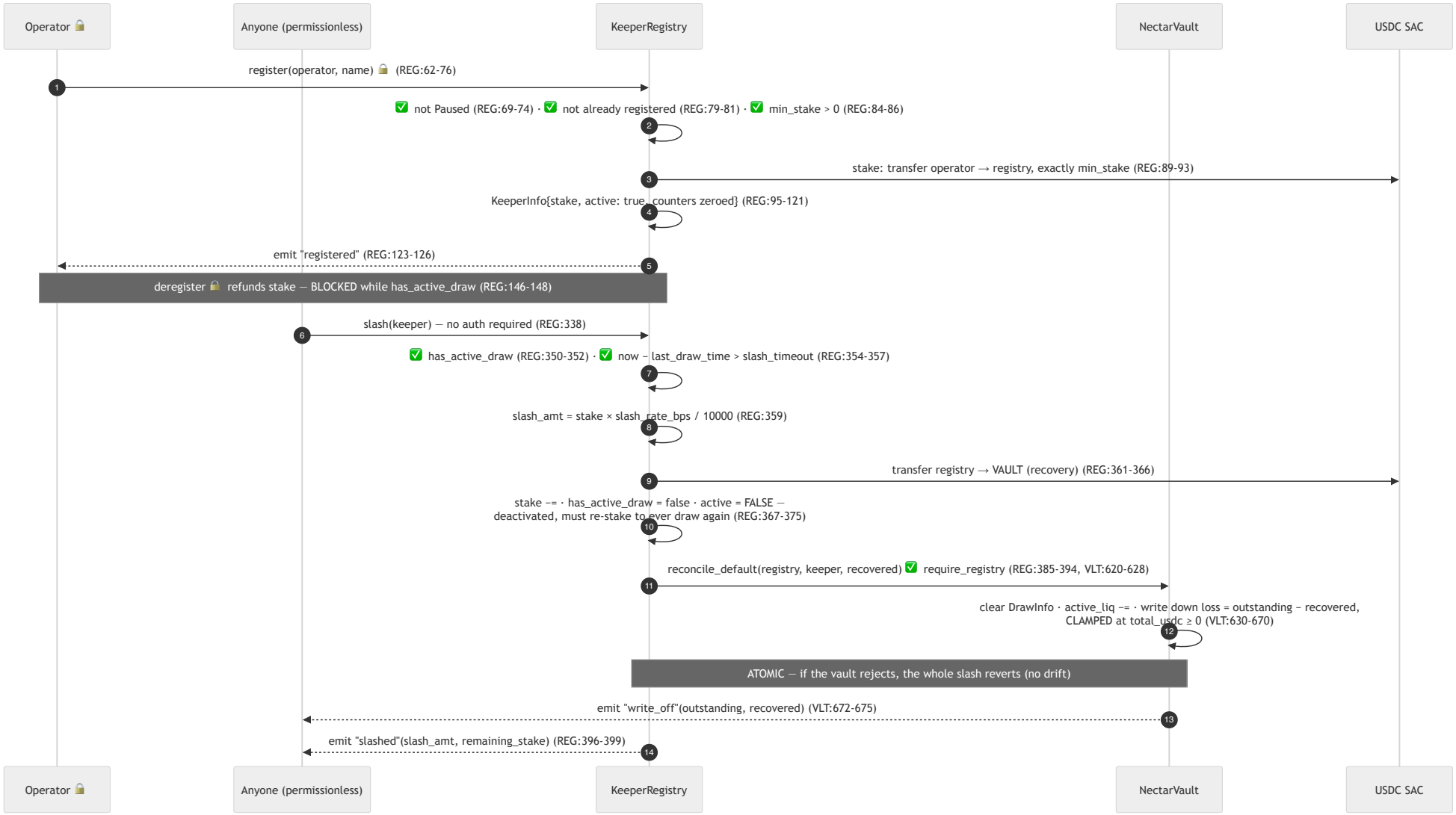
---

5. add_profit — registration-gated donation (shipped at this tag)

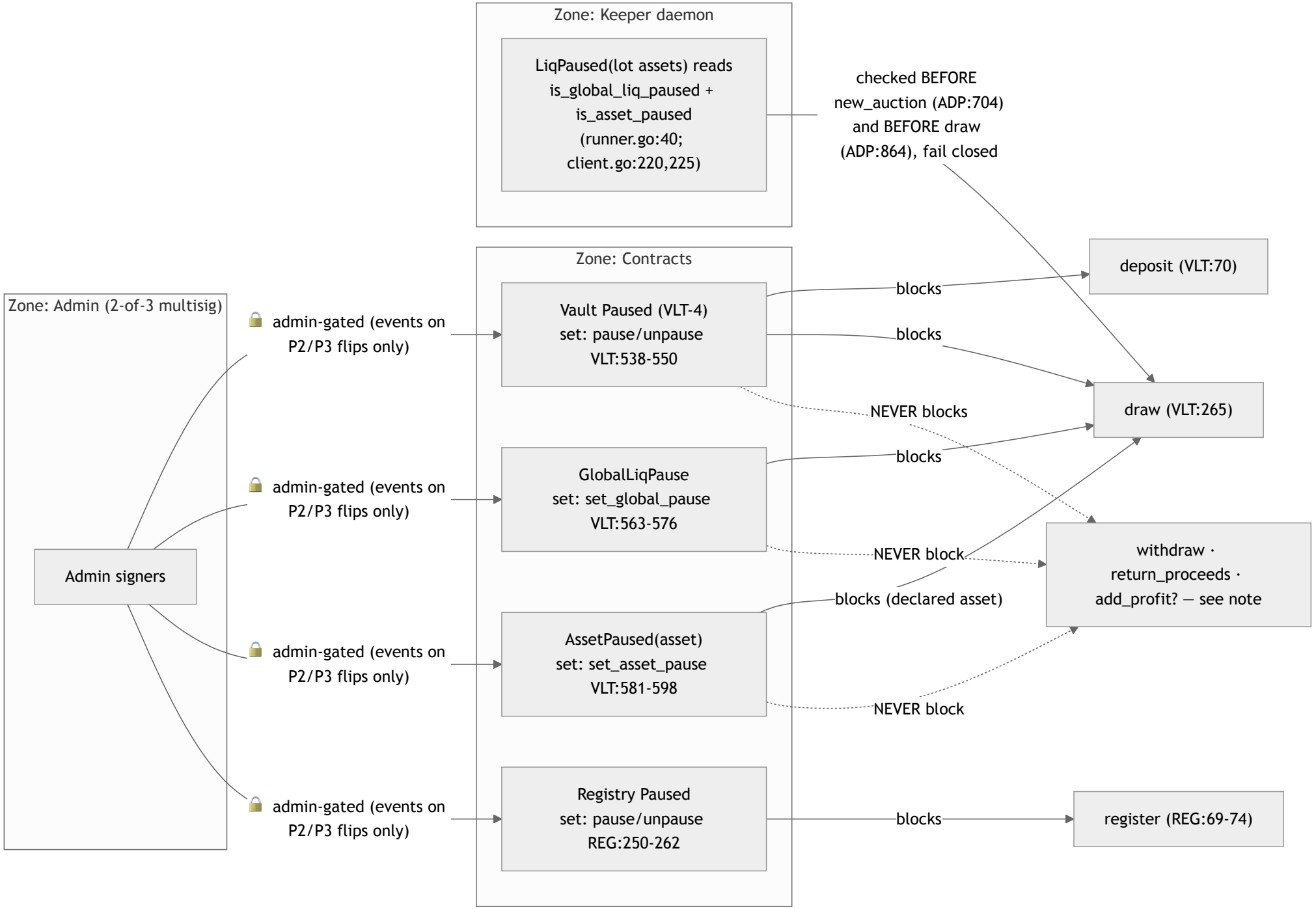
This is the deliberate, gated counterpart to the path VLT-2's `NoDraw` guard blocks for anonymous callers in `return_proceeds`. Share price rises for existing holders only; the `profit_added` event keeps donated profit separable in accounting.

---

6. Registration, staking, slashing



7. Pause propagation — who sets, who reads, what stops


Zone: Keeper daemon
LiqPaused(lot assets) reads is_global_liq_paused + is_asset_paused (runner.go:40; client.go:220,225)
checked BEFORE new_auction (ADP:704) and BEFORE draw (ADP:864), fail closed
Zone: Contracts
Zone: Admin (2-of-3 multisig)
Admin signers
admin-gated (events on P2/P3 flips only)
admin-gated (events on P2/P3 flips only)
admin-gated (events on P2/P3 flips only)
admin-gated (events on P2/P3 flips only)
Vault Paused (VLT-4) set: pause/unpause VLT:538-550
GlobalLiqPause set: set_global_pause VLT:563-576
AssetPaused(asset) set: set_asset_pause VLT:581-598
Registry Paused set: pause/unpause REG:250-262
blocks
blocks
NEVER blocks
blocks
NEVER block
blocks (declared asset)
NEVER block
blocks
deposit (VLT:70)
draw (VLT:265)
withdraw · return_proceeds · add_profit? — see note
register (REG:69-74)

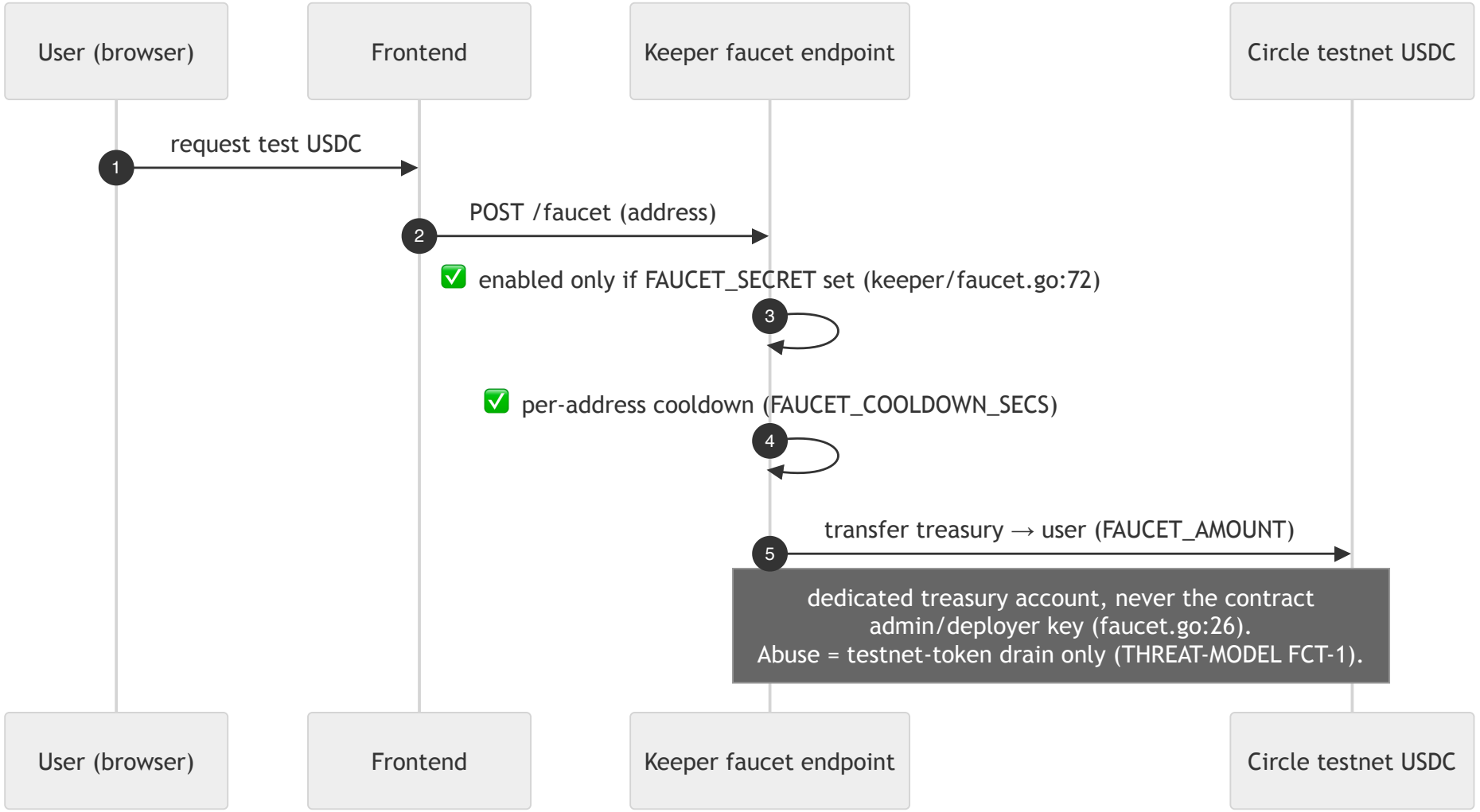
Exact gating at the tag — who reads each flag and what it stops. Note the event asymmetry: the two liquidation flags emit on every flip (`liq_pause`, `asset_pause`); the VLT-4 pause, registry pause and `set_config` do NOT emit (Scout `storage_change_events`, accepted — state remains readable via `is_paused`/getters):

Flag	Set by	Read on-chain at	Stops	Never stops
`Paused` (VLT-4)	admin `pause`/`unpause` (VLT:538-550)	`deposit` (VLT:70), `draw` (VLT:265) via `require_not_paused` (VLT:724-734)	deposit, draw	**withdraw** (no read in path), return_proceeds, add_profit, reconcile_default
`GlobalLiqPause`	admin `set_global_pause` (VLT:563-576, event `liq_pause`)	`draw` only, via `require_liq_allowed` (VLT:266, 736-746)	ALL draws regardless of declared asset	everything else — depositor exit invariant (VLT:559-562)
`AssetPaused(asset)`	admin `set_asset_pause` (VLT:581-598, event `asset_pause`)	`draw` only (VLT:747-755), against the keeper-DECLARED asset	draws declaring that asset (honest keepers hard-gated; misdeclaration = THREAT-MODEL T3-1)	everything else
Registry `Paused`	admin `pause`/`unpause` (REG:250-262)	`register` (REG:69-74)	new registrations	deregister, slash, vault-only calls

The keeper additionally reads the two liquidation flags off-chain for ALL lot assets and fails closed on read errors — before creating an auction and again before committing capital (ADP:704, 864) — so a paused system stops producing on-chain noise it can never act on.

---

8. Faucet (TESTNET ONLY — does not exist on mainnet)



Data entities crossing boundaries

Entity	Lives in	Crosses a boundary at
USDC	SAC balances (external zone)	deposit, withdraw, draw, return, add_profit, stake, slash recovery, faucet (testnet)

Entity	Lives in	Crosses a boundary at
Shares	Vault persistent `Depositor`	deposit (mint), withdraw (burn) — never transferable
`DrawInfo{amount, since}`	Vault persistent	draw (+/stamp), partial return (amount only — `since` preserved), full return / reconcile (remove)
`active_liq`	Vault instance `VaultState`	draw (+), return (–, capped per-keeper), reconcile (–)
Rate-limit window (`window_start`, `withdrawn_in_window`)	Vault persistent `Depositor`	withdraw only, and only while `max_withdraw_per_24h > 0`
Pause flags	Vault/registry instance	set by admin; read by draw/deposit/register paths + keeper (off-chain)
Stake	Registry persistent `KeeperInfo`	register (+), deregister (refund), slash (– → vault)
`has_active_draw` / `last_draw_time`	Registry `KeeperInfo`	mark_draw (set), clear_draw (unset), slash (unset + deactivate)
Performance counters	Registry `KeeperInfo`	record_execution (vault-only, saturating)
Positions / auctions / prices	Blend + oracle (external zones)	read by the keeper only — never enter either contract

External trust zones — what they can and cannot do

Blend, Soroswap, the comet, and the oracle are outside the Nectar boundary. The keeper is the only component that talks to them, and nothing they return can move vault funds: the vault's only inputs are keeper-signed calls that pass B2's checks (registration, caps, pauses) and admin-signed config. Oracle risk at this tag is therefore keeper-capital risk, not vault-solvency risk; the on-chain cross-referencing breaker (actuating these pause flags) is the Tranche-3 ORA-1 deliverable.

Part 2 — STRIDE Threat Model

Nectar Network — STRIDE Threat Model (audit-freeze-v1)

Scope: the two frozen Soroban contracts (`KeeperRegistry`, `NectarVault`) at tag `audit-freeze-v1` (commit `dbf0e5cd0a0c11bd123643fe72cf45ca2f35ccf5`), their cross-contract surface, the off-chain Go keeper daemon, and the admin surface. Prepared for the SCF Soroban Security Audit Bank.

Version: 2026-08-16 (Session G — supersedes the 2026-06-22 pre-hardening model) · **Network:** Stellar testnet today, mainnet in Tranche 3 · **Functional LOC:** 1,071 Rust code lines (tokei 14.0.0, excl. tests; NectarVault 672, KeeperRegistry 399; 1,438 raw lines incl. comments/blanks — see FREEZE-NOTE.md).

Method: the Stellar threat-modeling guidance (fetched 2026-08-16) — its four questions ("What are we working on?", "What can go wrong?", "What are we going to do about it?", "Did we do a good job?") with STRIDE per component. Threat IDs keep this repo's established series (VLT-x, REG-x, ORA-1, DEX-1, NEW-x, T3-x) for cross-document consistency with SCOUT-REPORT.md, FREEZE-NOTE.md and CORRECTION-REPORT.md; each row carries its STRIDE class.

Line references (`VLT:` = `contracts/nectar-vault/src/lib.rs`, `VLT-t:` = `contracts/nectar-vault/src/types.rs`, `REG:` = `contracts/keeper-registry/src/lib.rs`) are **at the frozen tag** — the working tree these were read from is byte-identical to `audit-freeze-v1` for `contracts/`.

> Candor note: this model imports the residual limitations from CORRECTION-REPORT.md verbatim rather than restating them favorably. Nothing below is invented for volume; nothing real is omitted for optics.

---

1. What are we working on? (System overview)

Nectar Network is a pooled liquidation protocol: depositors pool USDC in a vault, registered keeper operators draw that capital to fill Blend Protocol liquidation auctions, and measured profits return to the pool as depositor yield.

Components

Component	Type	Role at the frozen tag
NectarVault	Soroban contract (in scope)	Holds pooled USDC; mints/burns shares (virtual-offset math); lends capital via `draw(keeper, amount, asset)`; books profit on `return_proceeds`; accepts registration-gated donations via `add_profit`; enforces deposit cap, withdraw cooldown, per-address 24h rate limit, per-keeper cumulative draw cap, emergency pause, global + per-asset liquidation pause.
KeeperRegistry	Soroban contract (in scope)	Operator registration with a USDC stake bond (`min_stake`, enforced > 0); performance counters; permissionless timeout-gated `slash` that deactivates the keeper and atomically reconciles the vault.
Keeper daemon	Off-chain Go (out of audit scope)	Monitors Blend pools via an event-driven borrower index, sizes liquidations by on-chain simulation, checks the vault pause flags before creating auctions and before drawing (fail closed), executes ONE atomic fill+repay+withdraw, swaps collateral under an oracle-anchored floor, returns proceeds. Stateless except the borrower cache.

Component	Type	Role at the frozen tag
Frontend	Next.js on Vercel (out of scope)	Read-only dashboard + Freighter-signed deposit/withdraw. Holds no keys, no privileged authority.
USDC token	Stellar Asset Contract (external)	Circle testnet USDC today; Circle mainnet USDC at T3. Not our code.
Blend pool + backstop + comet	External protocol	Liquidation auction source (two-phase 400-ledger Dutch auctions). Audited upstream: Code4rena Blend V2 + Certora formal verification (Feb–Mar 2025).
Soroswap router	External protocol	Collateral → USDC conversion during unwind.
Price oracle	External	Testnet: Blend's admin-settable `oraclemock` (NOT Reflector — wasm-hash proven, FACTS.md). Mainnet T3: real Reflector feed + circuit breaker (ORA-1, not yet built).
Faucet	Keeper HTTP endpoint (testnet-only)	Dispenses test USDC from a dedicated treasury key with per-address cooldown (`keeper/faucet.go`). Does not exist on mainnet.

Assets to protect

1. **Pooled vault USDC** (`VaultState.total_usdc`) and the `active_liq` liability ledger.
2. **Keeper stakes** held by the registry.
3. **Share-accounting integrity** — the `total_usdc/total_shares` ratio is every depositor's claim; inflation or underflow of either side is direct theft/loss.
4. **Pause/admin controls** — the VLT-4 emergency pause, the T3 liquidation circuit-breaker flags, and the config parameters (caps, cooldown, rate limit, slash economics).
5. **Depositor exit liveness** — the invariant that a liquidation pause NEVER blocks `withdraw` (VLT:559-562 doc + `require_liq_allowed` being absent from the withdraw path, VLT:137-226).

Actors

Actor	Trust level	Capabilities
Depositor	Untrusted	`deposit`/`withdraw` own funds only (`user.require_auth()`).
Keeper operator	Untrusted but **bonded**	`register` (stakes `min_stake`), `draw` (verified + capped), `return_proceeds`, `add_profit`, `deregister` (blocked while a draw is outstanding). Misbehavior backstop: slashing + deactivation.
Admin	Trusted, **2-of-3 multisig**	`set_config`, `pause`/`unpause`, `set_global_pause`, `set_asset_pause`, registry `set_config`/`pause`/`set_vault`. Multisig enforcement verified live at the classic auth layer (`docs/evidence/f5-multisig.md`: 1 sig rejected `OpBadAuth`, any 2 of 3 accepted).
Anyone (anonymous)	Untrusted	`slash(keeper)` — permissionless by design, gated by `has_active_draw` + `slash_timeout` (REG:350-357). Read-only getters.
External protocols	Own trust zones	Blend, Soroswap, oracle. The vault never reads them; only the keeper mediates, and the vault only ever sees keeper-signed USDC transfers.

2. Trust boundaries

Aligned 1:1 with the zones in DATAFLOW.md.

1. **B1 Depositor ↔ Vault** — `require_auth` + cap/cooldown/rate-limit checks (VLT:71, 89, 159, 191-200).
2. **B2 Keeper ↔ Vault** — `require_auth` + registry verification + pause gates + cumulative draw cap (VLT:262-311, 463-471).
3. **B3 Vault ↔ Registry (mutual)** — the registry accepts `mark_draw`/`clear_draw`/ `record_execution` only from the stored vault address (`require_vault`, REG:440-451); the vault accepts `reconcile_default` only from the stored registry address (`require_registry`, VLT:758-769). Both sides pin the counterparty at construction/one-time-set (`set_vault` is one-time, REG:51-60).
4. **B4 Admin ↔ Contracts** — identity check against stored admin THEN `require_auth` (VLT:711-722, REG:427-438); the signature policy itself lives at the Stellar account layer (2-of-3 multisig, verified live).
5. **B5 Keeper ↔ External protocols** — Blend/Soroswap/oracle are separate zones; nothing they return can move vault funds without a keeper-signed vault call that passes B2's checks.
6. **B6 Browser ↔ Frontend ↔ Keeper API** — read-only data + SSE log stream + the testnet faucet endpoint; no privileged path.
7. **B7 Anyone ↔ Registry (slash)** — permissionless entry, on-chain-gated.

---

3. What can go wrong? (STRIDE by component)

3.1 NectarVault

- **Spoofing** — every money-moving entry point (`deposit`, `withdraw`, `draw`, `return_proceeds`, `add_profit`) calls `require_auth()` on the acting address (VLT:71, 140, 267, 364, 466). Keeper legitimacy is an explicit cross-contract `verify_keeper` — registered AND active AND `stake >= min_stake` (REG:196-218, called at VLT:771-785) — the VLT-3 class (implicit existence-only check) is closed.
- **Tampering** — share math floors toward the pool on both sides with a symmetric virtual offset (VLT:15-28); donation-based inflation is economically irrational (VLT-1 defense) and proven unprofitable by a 2000-case property test. `return_proceeds` caps principal repayment at the calling keeper's own outstanding draw so one keeper's return can never clear another's liability (VLT:396-404). `withdraw` clamps payout at `total_usdc` and `reconcile_default` clamps the loss write-down at zero — both preventing a persisted negative `total_usdc` (VLT:174-185, 659-667).
- **Repudiation** — every state change emits an event: `deposit`, `withdraw`, `draw` (now including the declared asset), `return`, `profit_added` (distinct from draw-cycle profit by design), `write_off`, `liq_pause`, `asset_pause` (VLT:129, 221, 346, 439, 497, 573, 595, 672).
- **Information disclosure** — no secrets on-chain; all state deliberately public.
- **Denial of service** — `draw` is bounded by available capital and the CUMULATIVE per-keeper cap (VLT:292-309). The admin can pause deposits+draws (VLT-4) and liquidation draws globally/per-asset (F-2a) — but **cannot pause** `withdraw`: no pause flag is read anywhere in the withdraw path, so depositor exit survives every admin state (the withdraw-during-pause invariant is pinned by test). The withdrawal rate limit (VLT:191-200) is itself a DoS-shaped control — see T3-2 for the abuse analysis in both directions.
- **Elevation of privilege** — `set_config`/pauses are admin-gated (identity check before `require_auth`, VLT:711-722); `reconcile_default` is registry-only (VLT:758-769). A keeper cannot reach any admin function.

3.2 KeeperRegistry

- **Spoofing** — `register`/`deregister` require the operator's auth (REG:76, 139); vault-only entry points pin the caller to the one-time-set vault address (REG:440-451); admin functions pin the stored admin (REG:427-438).
- **Tampering** — performance counters use `saturating_add` (REG:319-325); stake moves only via `register` (in), `deregister` (refund, blocked while `has_active_draw`, REG:146-148), `slash` (to the vault). `slash_rate_bps` is capped at 10,000 and `min_stake > 0` is enforced in BOTH the constructor and `set_config` (REG:29-39, 407-414) — no config state can mint a zero-bond keeper, which the vault's `add_profit` gate relies on.
- **Repudiation** — `registered`, `deregistered`, `draw_marked`, `draw_cleared`, `execution`, `slashed` events cover every transition.
- **Information disclosure** — none; metrics are public by design.
- **Denial of service** — `register` respects the `Paused` flag (REG:69-74). The keeper list is an unbounded `Vec<Address>` — growth-griefing needs ~3,300 staked registrations (~330k USDC locked), triaged as economically absurd (SCOUT-REPORT, `dynamic_storage`, accepted).
- **Elevation of privilege** — `slash` deactivates the keeper (REG:369-375), so a slashed operator cannot re-draw the cap each timeout window (NEW-drain closed); re-registering costs a fresh full stake.

3.3 Keeper daemon (off-chain, out of audit scope — modeled for completeness)

- **Spoofing** — holds `KEEPER_SECRET`; compromise exposes that operator's stake and outstanding draw (bounded by `max_draw_per_keeper` + slashing), never the pool directly.
- **Tampering/DoS** — stateless against chain state each cycle; restarts safely. Pause flags are read before creating auctions AND before drawing, failing closed on read errors (`keeper/adapters/blend/adapter.go:704, 864`). It cannot bypass any contract check regardless of bugs.
- **Repudiation** — every on-chain action is a signed transaction; the daemon also keeps structured logs + Prometheus counters (e.g. `nectar_cycle_overruns_total`).
- **Elevation** — none available; contracts are the source of truth.
- Known operational limits (imported, not hidden): one process per keeper address (chain-derived stale-draw recovery cannot tell a crashed sibling from an in-flight one — observed live, no funds lost); cycle time can overrun the 10 s poll interval (11–12 s observed, `LoadPool` dominates); borrowers idle past the ~7-day RPC retention window are reachable only via the borrower cache or `WATCH_ADDRESSES`.

3.4 Cross-contract surface (Vault ⇄ Registry)

- The draw path is consistency-locked: `draw` rejects `amount <= 0` so the vault-side `DrawInfo.since` and the registry's `last_draw_time` always move together (VLT:269-279, 340-344 — the zero-amount desync was a freeze-review finding, fixed pre-tag). Partial returns preserve `since` so a 1-stroop return cannot push out slash eligibility (VLT:412-430).
- `slash` and `reconcile_default` are atomic: the registry cross-calls the vault inside `slash` and a vault rejection reverts the whole slash (REG:381-394), so vault and registry accounting never drift (NEW-reconcile closed).
- Neither contract ever calls an address supplied by an untrusted caller: the vault invokes only its stored registry (VLT:771-828) and the registry only its stored vault (REG:343) — no confused-deputy path.

3.5 Admin surface (incl. the new T3 controls)

- **Pause-flag abuse (malicious/compromised admin):** the worst an admin can do with the T3 flags is stop NEW liquidation draws (`set_global_pause`, `set_asset_pause` — VLT:563-598) and, with the VLT-4 pause, stop deposits+draws. Withdraw and `return_proceeds` are exempt from every flag by construction, so an admin cannot trap depositor funds or prevent keepers settling debts. The two liquidation flags emit an event on every flip (`liq_pause`, `asset_pause`); the VLT-4 pause and `set_config` do not emit (Scout `storage_change_events`, accepted) but their state is readable on-chain (`is_paused`, `get_config`), so throttling is always detectable, if not always evented.

- **Config abuse:** `set_config` can re-parameterize caps/cooldown/rate-limit (VLT:518-533) and slash economics (REG:404-417, bounded: rate ≤ 100%, min_stake

  > 0). A hostile config (e.g. `max_withdraw_per_24h = 1`) can throttle exits to a crawl — this is the accepted residual admin-trust risk that the 2-of-3 multisig (and a T3 timelock candidate) bounds; it cannot steal funds directly.

- **Multisig key loss/compromise:** enforcement is at the Stellar account layer (verified live — `docs/evidence/f5-multisig.md`). Loss of ONE key of three keeps full control (any 2 of 3 sign; the master key holds no extra privilege beyond its weight); the operational procedure is to rotate the lost key out via a medium/high-threshold `set_options` signed by the two remaining keys. Compromise of TWO keys is full admin compromise — bounded by the "what can admin actually do" analysis above (throttle, pause, re-parameterize — not confiscate). The signing runbook (build → simulate → per-signer sign → send) is recorded in f5-multisig.md.

---

4. Threat table

Severity: impact on depositor/keeper funds assuming the mitigation were absent. Status/mitigation cites the frozen tag. "Residual" is what remains WITH the mitigation in place.

Per-letter STRIDE index (every category has at least one tabled issue):

- **S**poofing: VLT-2, VLT-3, NEW-init, T3-1, FCT-1 · **T**ampering: VLT-1, VLT-2, VLT-6, NEW-cap, NEW-reconcile, T3-2, T3-3, T3-5, ORA-1, DEX-1, INT-1, FR-1
- **R**epudiation: REP-1 · **I**nformation Disclosure: INF-1
- **D**enial of Service: VLT-4, REG-1, T3-2, T3-4, T3-6, T3-7, FCT-1 · **E**levation of Privilege: VLT-5, NEW-drain, NEW-init, T3-6

ID	Component	STRIDE	Threat	Severity	Mitigation (at `audit-freeze-v1`)	Residual risk
VLT-1	Vault	Tampering	Share-inflation / first-depositor attack: donate to inflate share price so a victim's deposit mints 0 shares.	High	Symmetric virtual offset `VIRTUAL_OFFSET = 1_000_000` in `to_shares`/`to_assets` (VLT:8-28) + zero-share deposits rejected (VLT:96-100). Attack proven money-losing by the 2000-case property suite (share math, `prop_test.rs`) plus a dedicated unit test with `add_profit` as the vector (`test_add_profit_inflation_attack_still_unprofitable`, test.rs:1507).	Offset-scale rounding dust; none profitable.
VLT-2	Vault	Spoofing/Tampering	Anonymous donation booked as profit via `return_proceeds` (share-price distortion + metric pollution).	Medium	`NoDraw` guard: returns require an outstanding draw (VLT:366-381). The legitimate donation need is met by `add_profit`, which is registration-gated (see T3-3).	None known.
VLT-3	Vault	Spoofing	Keeper verification implicit/incomplete (record-existence only).	Medium	Explicit `verify_keeper`: registered AND active AND `stake >= min_stake` (REG:196-218; invoked VLT:771-785; `min_stake > 0` invariant REG:29-39, 407-414).	None known.
VLT-4	Vault	DoS	No emergency stop during an incident.	Medium	Admin `pause`/`unpause` gate deposit+draw; withdraw and `return_proceeds` stay open (VLT:535-557, 724-734).	Admin trust for the flag itself (see T3-4).

ID	Component	STRIDE	Threat	Severity	Mitigation (at `audit-freeze-v1`)	Residual risk
VLT-5	Both	Elevation	Single admin key compromise re-parameterizes the protocol.	Medium	2-of-3 classic multisig on the admin account, enforced at the classic auth layer BEFORE contract logic — verified live with reject/accept probes (`docs/evidence/f5-multisig.md`).	2-of-3 collusion/compromise; no on-chain timelock yet (T3 candidate). Note: the live probe used an intermediate Session-F build — it proves the auth mechanics, not the final byte code.
VLT-6	Vault	Tampering	Reentrancy/CEI ordering around token transfers.	Low	Effects-before-interaction in `withdraw` and `draw` (VLT:202-219, 313-338); Soroban's execution model + trusted Circle SAC as defense-in-depth.	Trusted-token assumption, documented.
NEW-cap	Vault	Tampering	Per-keeper cap enforced per-call, loopable to drain available pool.	High	Cap bounds CUMULATIVE outstanding draw (VLT:297-309).	None known.
NEW-reconcile	Cross	Tampering	Slash without vault write-off leaves phantom assets backing shares.	High	`slash` atomically cross-calls `reconcile_default`; vault rejection reverts the slash (REG:381-394; VLT:620-677).	Recovered USDC exceeding the clamped write-down stays unaccounted (conservative, donation-like) — documented FREEZE-NOTE "known properties".
NEW-drain	Registry	Elevation	Slashed keeper re-draws the full cap every timeout window, losing only `slash_rate` each time.	High	`slash` deactivates (`active = false`, REG:369-375); re-drawing requires a fresh full stake.	Economics scale with `min_stake` vs `max_draw_per_keeper` — parameter choice, flagged for audit review.
NEW-init	Both	Spoofing/Elevation	Initialize front-run seizes admin on a fresh deploy.	Medium	Atomic `__constructor` on both contracts (VLT:35-64; REG:19-45); registry⇄vault link via one-time admin-gated `set_vault` (REG:51-60).	None known.
REG-1	Registry	DoS	Permissionless `slash` griefing.	Info	By design: gated by `has_active_draw` + `now - last_draw_time > slash_timeout` (REG:350-357); one slash per draw.	An honest-but-slow keeper past timeout can be slashed by anyone — intended economics.
T3-1	Vault	Spoofing	**Self-declared draw asset** (DECISION F-2a): a malicious keeper misdeclares `asset` to route around a PER-ASSET pause.	Medium	Documented limitation, not a bug: declaration is not on-chain-verifiable. Backstops: the GLOBAL pause blocks every draw regardless of declaration (VLT:736-756); the honest path is closed keeper-side (ALL lot assets checked, fail closed — `adapter.go:704, 864`); a misdeclaring keeper that defaults is slashed + deactivated.	**Accepted**: a bonded keeper can burn its stake to draw against a per-asset pause once. Sized by `max_draw_per_keeper` vs stake.

ID	Component	STRIDE	Threat	Severity	Mitigation (at `audit-freeze-v1`)	Residual risk
T3-2	Vault	DoS / Tampering	**Rate-limit bypass via multi-address sybil**: split funds across addresses to exceed the per-address 24h cap.	Low	Fixed-window per-address accounting (VLT:187-200; VLT-t:10-14). Sybil-splitting is **accepted by design** — the limit is a bleed-rate brake for a compromised-key scenario, not a global exit throttle; the cost of the control staying per-address is that it cannot bound aggregate outflow. Boundary behavior (~2× across a window rollover instant) is inherent to fixed windows, chosen by spec, review-verified.	Accepted (both the sybil path and the 2× boundary).
T3-3	Vault	Tampering	`add_profit` **as an attack vector**: donation-driven share-price manipulation or profit-laundering through the new entry point.	Medium	Registration gate: donor must pass `verify_keeper` (bonded, active, `stake >= min_stake > 0`) (VLT:463-471); `amount > 0` (VLT:468-470); empty-vault donations rejected (`NoShares`, VLT:478-484 — freeze-review finding, prevents permanently stranding funds on the phantom offset); VLT-1 offset math keeps inflation unprofitable regardless (2000-case property suite on the share math + the dedicated `add_profit`-vector unit test, test.rs:1507); distinct `profit_added` event keeps donated profit separable from draw-cycle profit. Not pause-gated by design: money in is always safe.	A bonded keeper can still donate to *raise* everyone's share price (harmless by construction — no shares minted, benefits existing holders only).
T3-4	Admin	DoS	**Pause-flag admin abuse**: compromised/hostile admin freezes the protocol.	Medium	Withdraw + `return_proceeds` are exempt from EVERY pause flag (no flag is read in either path; liq flags gate `draw` only — VLT:736-756, 559-562); every flip emits an event; admin is 2-of-3 multisig.	Deposits/draws can be halted and configs re-tuned by a compromised quorum; exits can be *throttled* (not stopped) via a hostile `max_withdraw_per_24h`. Timelock is the named T3 hardening candidate.
T3-5	Cross	Tampering	Zero/negative-amount `draw` desyncs vault `DrawInfo.since` from registry `last_draw_time` (walks slash eligibility).	Medium	`InvalidAmount` on `amount <= 0` (VLT:269-279); `mark_draw` therefore always paired with a real draw (VLT:340-344); partial returns preserve `since` (VLT:412-430). Freeze-review finding, fixed pre-tag.	None known.
T3-6	Admin	Elevation/DoS	Multisig key loss (lockout) or compromise.	Medium	2-of-3: one lost key ⇒ remaining two rotate it out (procedure in `docs/evidence/f5-multisig.md`); one compromised key alone can sign nothing.	Two simultaneous compromises = full admin (bounded by T3-4's "cannot confiscate" analysis); two simultaneous losses = admin lockout (config frozen; user funds still withdrawable — withdraw needs no admin).

ID	Component	STRIDE	Threat	Severity	Mitigation (at `audit-freeze-v1`)	Residual risk
ORA-1	Keeper/ext	Tampering	**Oracle manipulation → bad liquidation** (the Feb-2026 YieldBlox attack class): manipulated price makes a healthy position appear liquidatable or misprices collateral during unwind.	Med/High	Today (keeper-side only): liquidation sizing is delegated to on-chain simulation against the pool's own oracle; the unwind swap has an oracle-anchored floor that refused a ~30% adverse quote live (`docs/evidence/d-discovery.md` swap incident). The on-chain cross-referencing circuit breaker (auto-`set_asset_pause` on deviation) is the **Tranche-3 deliverable, not yet built**; the T3 pause flags at this tag are its actuation surface.	**Open until ORA-1 ships**: the vault itself has no oracle input (it never prices collateral), so vault solvency is not directly oracle-exposed — the exposure is keeper capital efficiency and Blend-side liquidation validity. Note: on testnet there is no Reflector to cross-reference (the pool oracle is a mock); the breaker targets mainnet feeds.
DEX-1	Keeper/ext	Tampering	Swap slippage / sandwich during collateral → USDC unwind.	Low/Med	Oracle-anchored min-out + on-chain `amount_out_min` (`SLIPPAGE_BPS`, default 1%); a sent-but-unknown swap is never blindly re-sent.	Sandwich within the tolerance band; keeper-capital risk, not vault-accounting risk.
T3-7	Keeper	DoS	**Stale-draw recovery abuse/failure**: capital stranded at a crashed keeper, or recovery selling assets it shouldn't.	Medium	Vault-side draw timestamps (`DrawInfo.since`, `get_keeper_draw → (amount, since)`, VLT:687-701) let a restarted keeper age its own stale draw from chain state alone (F4); recovery failures are surfaced, not silent (freeze-review fix); slashing is the protocol-level backstop for an unrecovered draw.	While a draw is outstanding, recovery treats the keeper's ENTIRE pool-reserve-token holding (above the XLM fee floor) as sellable — operators must not park personal funds on the keeper address (documented in CLAUDE.md/env docs). One-process-per-address rule (observed live).
FCT-1	Faucet	DoS/Spoofing	**Faucet abuse** (testnet-only): drain the test-USDC treasury via repeated claims.	Info (testnet-only)	Dedicated treasury account — never the contract admin/deployer key (`keeper/faucet.go:26`), per-address cooldown (`FAUCET_COOLDOWN_SECS`), fixed amount per claim; disabled when `FAUCET_SECRET` is unset. **Does not exist on mainnet.**	Sybil claims within cooldown limits can drain the test treasury — accepted; testnet funds only.
INT-1	Vault	Tampering	Integer overflow/underflow in share/profit math.	Low	i128 traps atomically in Soroban (no wraparound); products bounded ~10 orders below `i128::MAX` at cap-scale values; registry counters saturate; the two signed-underflow edges are clamped (withdraw VLT:174-185, reconcile VLT:659-667). Scout's overflow class triaged finding-by-finding (SCOUT-REPORT).	Out-of-band arguments self-revert (no-op).
FR-1	Vault	Tampering	Deposit/withdraw MEV around a large `return_proceeds`	Low	`withdraw_cooldown` (1 h default) makes deposit-snipe-exit non-atomic; floors favor the pool.	Timing yield-capture within cooldown bounds; noted for audit.

ID	Component	STRIDE	Threat	Severity	Mitigation (at `audit-freeze-v1`)	Residual risk
			(share-price front-run).			
REP-1	Both	Repudiation	An actor disputes an action (keeper denies a draw; depositor denies a withdrawal; admin denies a pause/config change).	Low	Every money-moving call is a signed transaction (non-repudiable at the protocol layer); every state transition emits an event (vault: `deposit`/`withdraw`/`draw`/`return`/`profit_added`/`write_off`/`liq_pause`/`asset_pause` — VLT:129, 221, 346, 439, 497, 672, 573, 595; registry: `registered`/`deregistered`/`draw_marked`/`draw_cleared`/`execution`/`slashed`). Known gap, stated: VLT-4 `pause`/`unpause` and `set_config` emit no event (Scout `storage_change_events`, accepted) — provable from signed tx history + readable state (`is_paused`, `get_config`), just not evented.	Admin config changes reconstructable via tx history only, not the event stream — accepted.
INF-1	Both	Info Disclosure	All contract state is public: depositor balances, cooldown timestamps, keeper stakes/metrics, and outstanding draws are enumerable; pending profitable liquidations are visible to competing searchers.	Info	By design on a public chain — no secrets or PII on-chain; keeper secrets exist off-chain only; the timing surface is covered by FR-1 (cooldown bounds deposit-snipe MEV).	Public-by-design; auction-target visibility is inherent to permissionless keeping — accepted.

---

5. What are we going to do about it? (Treatments)

All High findings and every confirmed finding from the pre-freeze adversarial review are **fixed at the tag** (see the table — VLT-1..4, VLT-6, NEW-cap/reconcile/drain/init, T3-3's guards, T3-5). The remaining open treatments, honestly stated:

1. **ORA-1 (open, Tranche 3):** build the oracle circuit breaker keeper-side module cross-referencing Reflector on mainnet, actuating `set_asset_pause`/ `set_global_pause`. The contract-side actuation surface is frozen at this tag; the breaker itself is new off-chain code + admin policy.
2. **T3-1 (accepted, documented):** self-declared draw asset — accepted with the stake-slash backstop; auditors are asked to review the economics (`max_draw_per_keeper` vs `min_stake`) rather than the mechanism.
3. **T3-4 residual (candidate):** config timelock to bound a compromised admin quorum's re-parameterization speed — T3 hardening candidate, not at this tag.
4. **Session H (keeper, out of contract scope):** wire the keeper's bad-debt LP unwind proceeds into `add_profit` (the contract path exists and is audited at this tag; the keeper does not call it yet), and redeploy the frozen contracts to testnet (the live pair predates this tag — FREEZE-NOTE "known properties").

6. Residual risks (imported from CORRECTION-REPORT.md — none omitted)

1. **Bad-debt profit reaches depositors only once Session H wires** `add_profit` — until then float-funded bad-debt profit accrues to the operator (CORRECTION-REPORT limitation 2; contract-side path RESOLVED at this tag, keeper-side wiring pending).

2. **LP unwind convexity:** the mainnet comet exit is one verified call but its cost is convex in size (0.24% at ~$5 → ~13% at ~1M LP, live-measured) and a single call caps at ≈ pool-USDC/3 — large lots need staged exits (limitation 3).
3. **Retention-window discovery bound:** borrowers idle > ~7 days (120,960 ledgers) are reachable only via the persisted borrower cache or `WATCH_ADDRESSES` (limitation 4; the cache is load-bearing, not an optimization).
4. **Cycle overrun:** keeper cycle time can exceed the 10 s poll interval (11–12 s observed; `LoadPool` dominates). Counted, correctness-independent, named follow-up exists (limitation 5).
5. **Wasm-reproducibility gap:** Blend's deployed pool wasm is cited against pinned source `ba22b48` but was not reproduced from source; divergence is theoretically possible (limitation 8).
6. **Interest auctions are never filled** (detect + defer, by design — limitation 1); **Blend's canonical testnet pool is monitor-only** (settles a different USDC — limitation 7); **emitter event shapes unverified** (limitation 10) — all keeper-scope, listed for completeness.
7. **Trusted-token assumption:** vault accounting is internal-state-based, but transfers assume a well-behaved (non-reentrant) USDC SAC — true for Circle USDC.
8. **The live testnet pair predates this tag** — the audited code is deployed fresh in Session H; "audited code == deployed code" holds at those deployments, not at the legacy pair (FREEZE-NOTE).

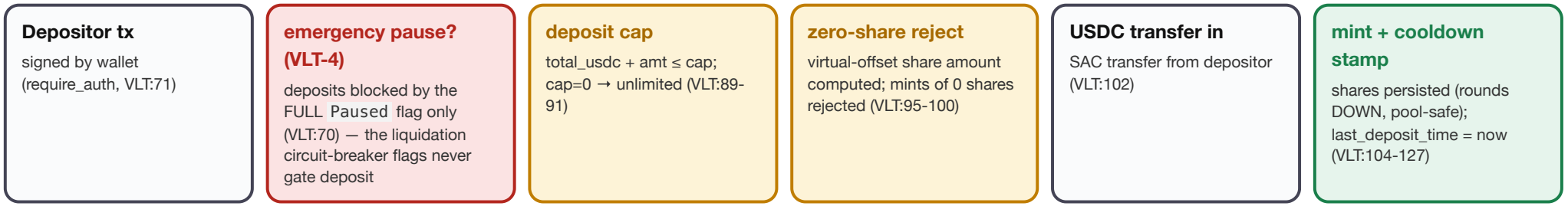
7. Did we do a good job? (Template question 4)

- The model was built by re-reading both frozen contracts end to end at the tag, not from prior drafts; every mitigation cite was checked against the frozen source.
- The pre-freeze adversarial review (21 agents, 16 findings: 9 confirmed → all fixed pre-tag, 7 refuted) exercised exactly the surfaces this model names; the freeze-review fixes (draw amount guard, reconcile clamp, add_profit empty-vault guard, keeper pause fail-closed, non-silent recovery) all appear above with cites.
- Adversarial functional coverage at the tag: inflation attack (unit + property, with and without `add_profit`), donation-assisted exit → slash write-off, pause/ exit invariants, rate-limit window boundaries (86399/86400), draw-timestamp consistency, zero/negative amounts on every money-moving function, cross-contract slash/reconcile against the real registry (FREEZE-NOTE suite table: 113 passing, 96 audit-scope).
- This is a living document: any audit remediation lands as `audit-freeze-v2` with this model updated in the same change.

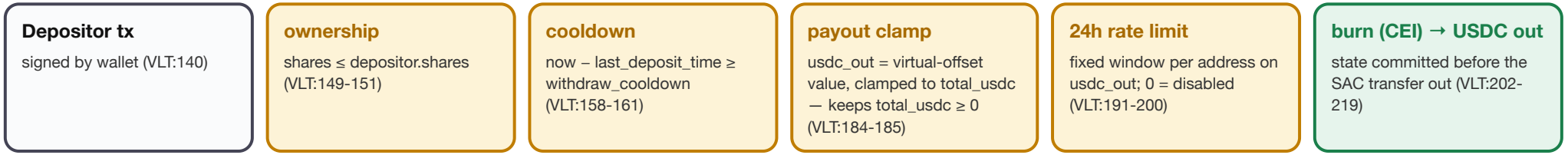
Appendix — One-glance summaries (code-anchored)

A. Depositor flows — on-chain gates, in enforcement order (NectarVault @ audit-freeze-v1)

DEPOSIT (VLT:67-134):



WITHDRAW (VLT:137-226):



INVARIANT (tested): the liquidation circuit-breaker flags (global / per-asset) gate draw() ONLY — they never block depositor withdrawals. Cooldown and the rate limit are the only contract gates on exit.

Liveness note (documented, not a gate): payout is sized from total_usdc, which counts drawn-but-unreturned capital — at high utilization the SAC transfer itself can fail for lack of liquid balance. A liveness bound, never fund loss and never a pause.

B. Full liquidation cycle — capital flow as shipped (evidence: docs/evidence/b-full-cycle.md)

1. Detect
event-driven borrower index + get_positions probe; HF < 1.0 at pool-oracle prices

2. Create auction
new_auction; percent chosen by read-only on-chain simulation (post-fill HF must land in [1.03, 1.15])

3. Wait on curve
two-phase Dutch: lot 0→100% (blk 0-200), bid 100→0% (blk 200-400); fill when lot/bid ≥ MIN_PROFIT

4. Draw capital
vault.draw(keeper, amt, asset) — pause flags, amount > 0, availability, CUMULATIVE per-keeper cap, verify_keeper all checked on-chain; registry.mark_draw stamps the clock

5. Atomic submit()
ONE tx: [fill, repay (gross), withdraw_collateral] — single health check at the end; either keeper exits with tokens and no debt, or nothing happened

6. Swap
Soroswap: seized collateral → vault USDC; oracle-anchored floor + on-chain amount_out_min

7. Return
vault.return_proceeds → clear_draw + record_execution; profit raises share price

Scope guard (verified): a pool whose settle asset differs from the vault's USDC is monitor-only — the adapter emits no tasks and Execute refuses (cross-USDC guard).

Failure discipline (demonstrated live, not injected): failure after step 4 → deterministic-failure bookkeeping + chain-derived stale-draw recovery on restart; in the observed incident capital returned to the vault with zero stroops lost. Protocol backstop regardless of keeper behavior: draw timeout → anyone may call registry.slash(keeper) — slashed stake transfers to the vault and the outstanding draw reconciles atomically.

Money test (testnet, tx-hashed): share price 1.0000000 → 1.0432672 across one full cycle.

C. System zones & trust boundaries — one glance

ACTORS (untrusted)

Depositor
: deposit / withdraw ·

Keeper operator
: staked, slashable ·

Admin
: 2-of-3 multisig (verified live), config + pause flags ·

Anyone
: slash() after timeout

ON-CHAIN — NECTAR (audit scope)

NectarVault
: deposit/withdraw · draw/return_proceeds · add_profit · caps · cooldown · rate limit · pauses · virtual-offset shares

KeeperRegistry
: register(stake) · mark/clear draw · record_execution · slash · perf metrics
cross-contract auth pinned both ways (require_vault / require_registry)

EXTERNAL ON-CHAIN (own trust zones)

USDC SAC
(Circle — not our code): vault asset + stake ·

Blend Pool v2
: submit(), 400-ledger Dutch auctions ·

Backstop/Comet
: bad-debt LOT = backstop LP; interest BID = backstop LP (detect + defer) ·

Oracle
: oraclemock on testnet (admin-settable); Reflector cross-ref is the T3 breaker deliverable (not built at this tag) ·

Soroswap
: unwind swaps, slippage-guarded

OFF-CHAIN (out of audit scope)

Keeper daemon (Go)
: event-driven discovery · HF monitor · profitability by on-chain sim · oracle-anchored swap floor (self-gates unwinds) · draws via its own keypair ·

Frontend
: wallet-signed txs only, no keys, no custody ·

Faucet (TESTNET ONLY)
: transfers 1,000 test USDC/claim from a dedicated treasury, per-address cooldown — does not exist on mainnet

Every zone crossing is validated on the far side: user auth at the vault; keeper legitimacy via verify_keeper (bonded + active + stake ≥ min_stake); vault-only and registry-only entry points pinned to stored addresses; nothing an external protocol returns can move vault funds without a keeper-signed call passing those gates.